



# Q4

## Part a

### General Overview

The logic for the serial code (found in `src/src_serial`) is as follows

- Load in the config file ( `config/config.txt` ) from the user
- Initialise all data types necessary
- Start a `while (true)` loop
- Within the loop, circle centers are generated via a seeded random number generator. Those circles are checked against an `unordered_map` of the other generated circles for overlap.
- If no overlap is found, it is placed within the `unordered_map`
- After the user parameter `sampling_frequency` number of trials, there is a check for convergence. If the number of circles generated has not changed after `sampling_frequency` trials, then there can be no more circles placed without overlap with these parameters.
- The final results are outputted and graphs can be plotted ( `graphing/graph.py` ).

The use of `make` with the rules `serial` , `serial_debug` , `serial_vis` are used to compile the source code. Each rule gives a different binary output with:

- `serial` : Runs the binary without any visualisation or debugging, and outputs final answer.
- `serial_debug` : Runs the binary with custom debugging flags.
- `serial_vis` : Runs the binary with custom visualisation flags, that continually outputs the key data for visualisation purposes (you can watch the numbers go up).

I highly recommend using the bash script `run_serial.sh` with command `bash ./run_serial <MAKE_RULE>` . This builds and executes the binary, and will create directories for results to be moved into, which then can be plotted using the Python

script.

## Specfics

### Generation of circles

```
/* Generates and returns a Point, within the boundaries */
Point gen_random_pair(rng &random_gen)
{
    return {
        r + (L - 2 * r) * random_gen.grnd(),
        r + (L - 2 * r) * random_gen.grnd(),
    };
}
```

The generating function will generate a random number taht has been rpreviously seeded, with a number that is within the boundaries of the box. This assures that no circle can overlap the boundary, and reduces the need to run four comparison checks every generation.

### Checking overlap

The biggest bottleneck to this problem is the checking the overlap of circles. A naive approach would be a simple comparison between every circle, giving an  $O(N^2)$  loop. For efficency, I implemented a hash map that relates `Cells` to `vector<Point>` i.e I split the grid up into cells that are determined by the input parameters `L` and `r` i.e. `grid_size = static_cast<int>(L / (2 * r));`.

```

// Check if a new circle overlaps with circles in nearby grid cells
bool check_overlap(const Point &new_circle)
{
    Cell cell = get_grid_cell(new_circle);

    const vector<Cell> neighbours = {{-1, -1}, {-1, 0}, {-1, 1},
    {0, -1}, {0, 0}, {0, 1}, {1, -1}, {1, 0}, {1, 1}};

    // Check nearby neighbour cells

    for (auto &neighbour : neighbours)
    {
        Cell neighbor_cell = {cell.first + neighbour.first,
        cell.second + neighbour.second};

        // Check if neighbor cell exists in the spatial grid
        if (grid.find(neighbor_cell) != grid.end())
        {
            for (const auto &existing_circle : grid[neighbor_cell])
            {
                // Compute squared distance for comparison
                double dx = existing_circle.first - new_circle.first;
                double dy = existing_circle.second - new_circle.second;
                double distance_squared = dx * dx + dy * dy;

                if (distance_squared < r_comp)
                {
                    return true; // If overlap, return true
                }
            }
        }
    }

    return false; // No overlap found
}

```

The `check_overlap` function will only compare the circles that are within the nearest neighbour cells. This is a lot more efficient than  $O(N^2)$ , as a hash map

(with a good hash) is on average  $O(1)$  lookup time.

## Sampling Frequency approach

I implemented a `sampling_frequency` approach to the generation of circles, to fit into the idea of "run until no more circles can be placed". Choosing a small `sampling_frequency` will be more efficient, however is very likely to miss a lot of circles that could be generated. On the other hand, choosing a large `sampling_frequency` will be less efficient, but you will likely catch a lot of circles that could be generated. An improvement that could be made is to make the parameter dynamic. I found that a parameter of `1e6` is a good benchmark, however results have been given for `1e6` , `2.5e6` , `5e6` , `7.5e6` , `1e7` . These can also be found in `results/serial_results/L=500, r=1.234/sf={SAMPLING_FREQUENCY}` . All these results were run on `VIKING` with `job_serial.sh` .

## Results